

Lec 1

Data Structures & Algorithms — Course Notes

Date: 13th July 2026

Course Policy — Unit Overview

Unit	Topic
Unit 1	Introduction
Unit 2	Recursive & Iterative Operations
Unit 3	Linear and Non-Linear Data Structures
Unit 4	Sorting Algorithms

1. General Introduction

1.1 Basic Concepts

Aim: Design a program that is efficient, correct, and maintainable.

1.2 Properties of a Good Program

A good program should:

- Run correctly
- Be easy to read and understand
- Be easy to modify and debug

1.3 Efficiency

Efficiency of a program depends on two factors:

1. **Time** — how fast it executes
2. **Space** — how much memory it consumes

Good **data management** is essential to achieving both.

1.4 Data Management

Data management involves:

- **Collecting** data
 - **Organising** it in an appropriate structure
 - **Maintaining** data and ensuring quality assurance
-

2. Data Structures — Definition

A **data structure** is a group of data elements put together under one name, which defines a particular way of storing and organising data in computer memory so that it can be used efficiently.

In simple terms: Organisation of data for efficient use.

2.1 Data Type vs Data Structure

Term	Definition
Data Type	Permitted values + operations allowed on those values
Data Structure	Collection of data elements + the permitted operations on that collection

We define rules to describe how data structures behave in memory. Storage and retrieval operations can be carried out on data held in **main memory (RAM)** and **secondary memory (disk)**.

3. Types of Data Types

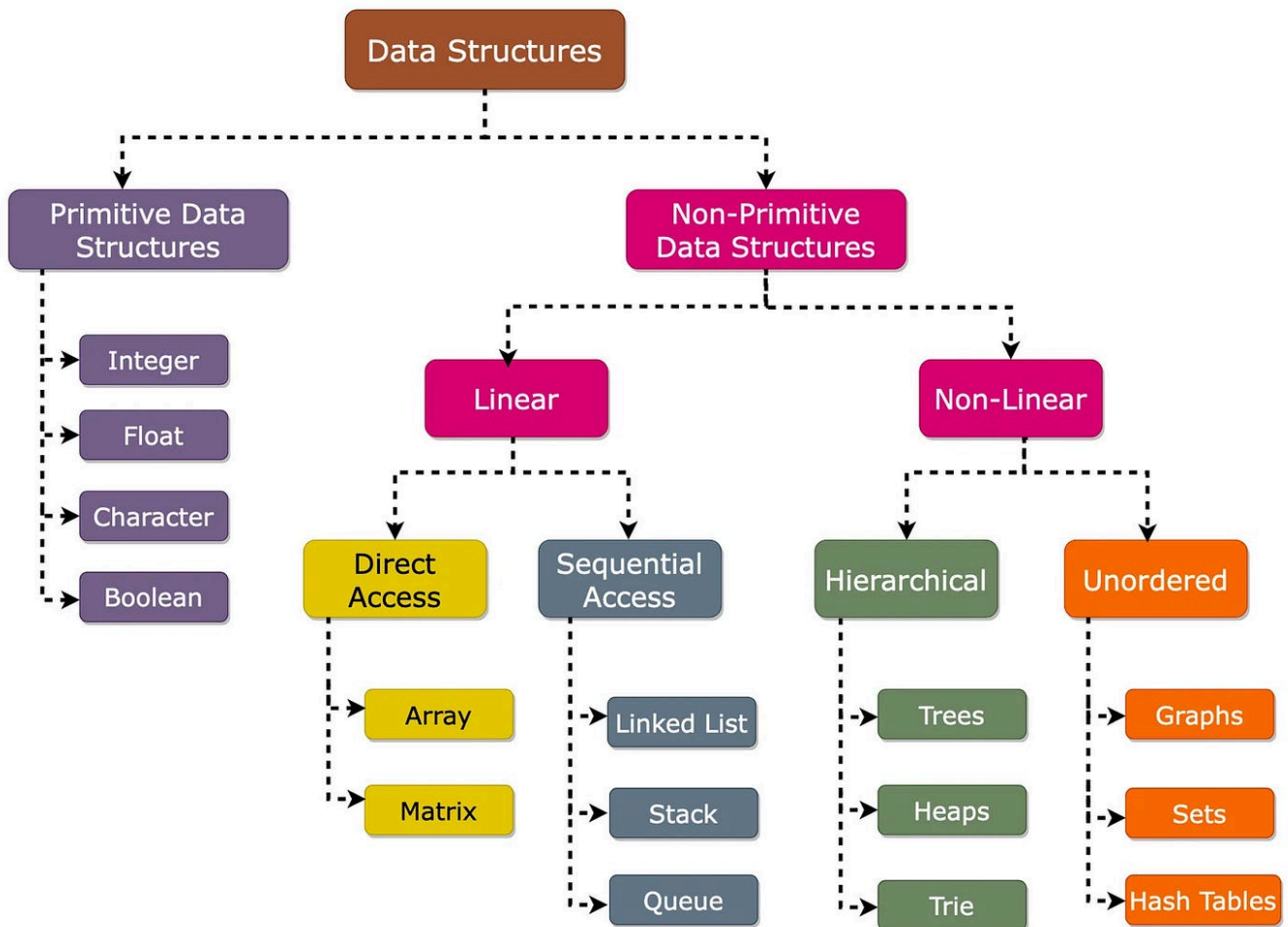
Data types are broadly classified as follows:

```
Data Types
├─ Primitive (Built-in)
│   ├── int
│   ├── float / double
│   ├── char
│   ├── boolean
│   └─ pointer
│
```

```

└─ Non-Primitive (Derived / User-defined)
    └─ Linear
        └─ Array
        └─ Linked List
        └─ Stack
        └─ Queue
    └─ Non-Linear
        └─ Tree
        └─ Graph
        └─ Hash Table / Map

```



3.1 Primitive Data Types

Basic, built-in data types provided directly by a programming language. They are **atomic** — cannot be broken down further.

- Directly operated upon by machine-level instructions
- Fixed, small memory footprint
- Examples: `int`, `float`, `char`, `boolean`, `pointer`

3.2 Non-Primitive Data Types

Derived from primitive data types; used to store a **collection** of values, either of the same type or different types.

- Can be broken down into primitive types
- User-defined structuring is possible
- Examples: Array, Linked List, Stack, Queue, Tree, Graph

4. Classification of Data Structures

4.1 Primitive vs Non-Primitive Data Structures

Aspect	Primitive Data Structure	Non-Primitive Data Structure
Definition	Basic structures directly operated on by machine instructions	Built using primitive structures; more complex organisation
Examples	int, float, char, pointer	Array, Linked List, Stack, Queue, Tree, Graph
Complexity	Simple	Complex
Derivation	Not derived from anything	Derived from primitive types

4.2 Linear vs Non-Linear Data Structures

Aspect	Linear	Non-Linear
Arrangement	Elements arranged sequentially, one after another	Elements arranged in a hierarchical / networked manner
Traversal	Single pass (one level)	May require multiple passes / levels
Examples	Array, Linked List, Stack, Queue	Tree, Graph
Memory utilisation	Generally less efficient for complex relationships	More efficient for representing hierarchical/relational data

5. Array vs Linked List

5.1 What is an Array?

An **array** is a collection of elements of the **same data type**, stored in **contiguous memory locations**, and accessed using an **index**.

Pros:

- Fast random access — $O(1)$ time using index
- Memory efficient (no extra pointer storage)
- Cache-friendly due to contiguous memory
- Simple to implement and use

Cons:

- Fixed size (in static arrays) — cannot grow/shrink dynamically
- Insertion/deletion is expensive — $O(n)$, requires shifting elements
- Wastage of memory if allocated size is larger than needed
- Contiguous memory requirement can cause allocation failure for large sizes

5.2 What is a Linked List?

A **linked list** is a linear data structure where elements (**nodes**) are stored in non-contiguous memory locations, and each node contains:

1. **Data**
2. **A pointer/reference** to the next node (and possibly the previous node in doubly linked lists)

Pros:

- Dynamic size — grows/shrinks at runtime
- Efficient insertion/deletion — $O(1)$ if position/node is known
- No memory wastage from over-allocation
- No need for contiguous memory

Cons:

- No random access — must traverse sequentially, $O(n)$ to access an element
- Extra memory required for storing pointers
- Not cache-friendly (nodes scattered in memory)
- More complex to implement (pointer management)

5.3 Side-by-Side Comparison

Feature	Array	Linked List
Memory allocation	Contiguous	Non-contiguous (scattered)
Size	Fixed (static) / costly to resize	Dynamic
Access time	$O(1)$ — direct indexing	$O(n)$ — sequential traversal
Insertion/Deletion	$O(n)$ — shifting required	$O(1)$ — if node/position known
Memory overhead	Low (only data)	Higher (data + pointer)
Cache performance	Good (contiguous)	Poor (scattered)
Use case	When frequent access/search is needed	When frequent insertion/deletion is needed

6. Quick Recap

- A **data structure** organises data for efficient use.
 - **Data type** = permitted values + operations; **data structure** = collection of values + operations.
 - Data structures are split into **primitive** (int, float, char) and **non-primitive** (array, linked list, stack, queue, tree, graph).
 - Non-primitive structures are further split into **linear** (sequential) and **non-linear** (hierarchical/networked).
 - **Arrays** favor fast access but costly insertion/deletion; **linked lists** favor fast insertion/deletion but slow access.
-

Notes compiled and formatted from lecture material dated 13th July 2026.